

Contexto do Sistema: Gestão de Biblioteca

Considere as seguintes entidades para os exercícios:

1. **Entidade Livro:** id, titulo, isbn, preco, dataPublicacao, categoria (Enum ou String), Editora.
2. **Entidade Autor:** id, nome, nacionalidade, dataNascimento.
3. **Relacionamento:** Um Livro possui muitos Autores (Many-to-Many).

Entidades de Referência

Java

@Entity

```
public class Livro {  
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String titulo;  
    private String isbn;  
    private BigDecimal preco;  
    private LocalDate dataPublicacao;  
    private String categoria;  
  
    @ManyToOne  
    private Editora editora;  
  
    @ManyToMany  
    @JoinTable(name = "livro_autores")  
    private List<Autor> autores;  
}
```

@Entity

```
public class Autor {  
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String nome;  
    private String nacionalidade;  
}
```

Exercícios de Fixação

Nível 1: Derived Query Methods (Padrão Spring Data)

1. **Find:** Crie um método que encontre livros pelo título exato.
2. **And:** Busque livros por categoria AND preco menor que um valor informado.
3. **Between:** Busque livros cujo preço esteja entre dois valores (min e max).

4. **Containing:** Busque autores cujo nome contenha uma determinada sequência de caracteres (Case Insensitive).
5. **In:** Busque livros que pertençam a uma lista de categorias (Ex: 'Terror', 'Ficção').
6. **IsNull/IsNotNull:** Busque livros que ainda não possuem um ISBN cadastrado.
7. **OrderBy:** Liste todos os livros de uma editora, ordenados pelo título de A-Z.
8. **Count:** Conte quantos livros existem de uma determinada nacionalidade de autor.

Nível 2: JPQL (Java Persistence Query Language)

9. **Select Simple:** Crie uma query JPQL que selecione apenas o título dos livros de uma categoria específica.
10. **Join:** Selecione livros onde o nome do Autor seja igual ao parâmetro passado.
11. **Join Fetch:** Resolva o problema do N+1 criando uma query que busque o livro e já traga seus autores carregados (fetch).
12. **Agregação:** Calcule a média de preço dos livros de uma determinada editora usando `AVG`.
13. **Subquery:** Busque livros cujo preço seja maior que a média de todos os livros cadastrados.

Nível 3: Native Queries (SQL Puro)

14. **Select Básico:** Faça uma query nativa para selecionar todos os campos da tabela `livro` onde a `data_publicacao` seja no ano de 2023.
15. **Join Nativo:** Realize um join nativo entre `livro`, `livro_autores` e `autor` para retornar livros de autores brasileiros.
16. **Função de Banco:** Utilize a função `LOWER` do SQL para buscar livros por categoria, ignorando maiúsculas e minúsculas via Native Query.

Nível 4: Projections e DTOs

17. **Interface Projection:** Crie uma interface `LivroMinimoProjection` (apenas título e preço) e um método no repository que a utilize.
18. **Class-based DTO:** Crie um DTO `EstatisticasEditoraDTO` e use `SELECT new ...` em JPQL para retornar o nome da editora e a contagem de livros dela.
19. **Native Projection:** Utilize uma Query Nativa para mapear o resultado para uma Interface Projection (lembre-se de usar aliases `as` que coincidam com os métodos da interface).
20. **Dynamic Projection:** Crie um método genérico no repository que aceite um parâmetro `<T> Class<T type>` para retornar diferentes tipos de Projections para a mesma consulta de livros.